



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Синхронизирующие примитивы

Направление подготовки бакалавров
«Фундаментальная информатика»



Синхронизирующие примитивы

Операционные системы



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Взаимодействие процессов

Проблема взаимодействия между процессами разбивается на три компонента:

- Передача информации от одного процесса к другому;
- Контроль над деятельностью процессов: гарантия того, что процессы не «пересекутся» в критических ситуациях;
- Согласование действий процессов для избежание тупиковых ситуаций.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Взаимодействие процессов

Ситуации, в которых два (и более) процесса считывают и записывают данные одновременно и конечный результат зависит от того, какой из них был первым, называют ***состояниями состязания (гонок)***.

Основным способом предотвращения проблем состязаний между процессами является запрет одновременной записи и чтения данных более чем одним процессом. Говоря иными словами, необходимо ***взаимное исключение***. То есть в тот момент, когда один процесс использует общие данные, другому процессу это делать запрещено.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Основные понятия синхронизации

Критическая секция (critical section) - это часть программы, в которой осуществляется доступ к разделяемым данным и результат выполнения которой может непредсказуемо меняться, если переменные, относящиеся к этой части программы, изменяются другими потоками в то время, когда выполнение этой части еще не завершено.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Основные понятия синхронизации

Критическая секция всегда определяется по отношению к определенным **критическим данным**, при несогласованном изменении которых могут возникнуть нежелательные последствия.

Чтобы исключить эффект гонок по отношению к некоторому ресурсу, необходимо обеспечить, чтобы в каждый момент времени в своей критической секции, связанной с этим ресурсом, находился максимум один процесс.

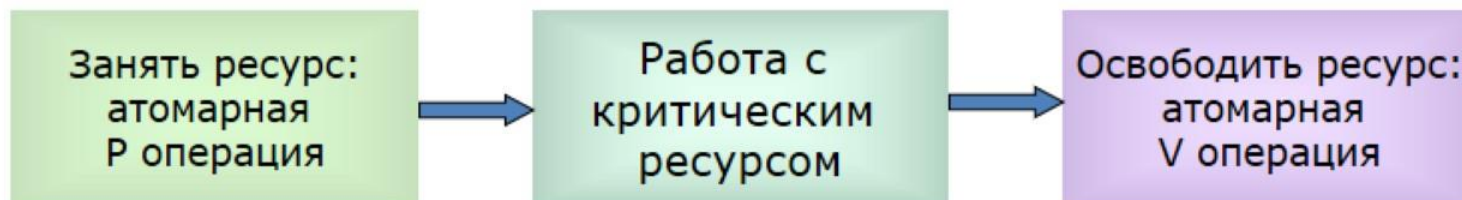


ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Семафоры

Семафоры – это тип переменных, значение которых может быть нулем (в случае отсутствия сохраненных сигналов активизации) или некоторым положительным числом, соответствующим количеству отложенных сигналов.

Алгоритм использования семафоров выглядит так:





ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

Дейкстра предложил две операции: $P(S)$ и $V(S)$.

- Операция $P(S)$ сравнивает значение семафора с нулем. Если значение семафора больше нуля, операция $P(S)$ уменьшает его (то есть расходует один из сигналов активации) и возвращает управление процессу. Если значение семафора равно нулю, процедура $P(S)$ не возвращает управление, а процесс переводится в состояние ожидания.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

Все операции проверки значения семафора $P(S)$, его изменения и перевода процесса в состояние ожидания выполняются как единое и неделимое (атомарное) элементарное действие. Тем самым гарантируется, что после начала операции ни один процесс не получит доступа к семафору до окончания или блокирования операции.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

- Операция $V(S)$ увеличивает значение семафора. Если с этим семафором связаны один или несколько ожидающих процессов, которые не могут завершить более раннюю операцию $P(S)$, то один из них выбирается системой (например, случайным образом) и ему разрешается завершить свою операцию $P(S)$. Таким образом, после операции $V(S)$, примененной к семафору, связанному с несколькими ожидающими процессами, значение семафора так и остается равным нулю, но число ожидавших процессов уменьшится на единицу.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

Семафор представляет собой целую переменную, принимающую неотрицательные значения, доступ любого процесса к которой, за исключением момента ее инициализации, может осуществляться только через две атомарные операции: P (от датского слова *proberen* – проверять) и V (от *verhogen* – увеличивать).



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

Классическое определение этих операций выглядит следующим образом:

$P(S)$: пока $S == 0$ процесс блокируется;

$S = S - 1$;

$V(S)$: $S = S + 1$;

Эта запись означает следующее: при выполнении операции P над семафором S сначала проверяется его значение. Если оно больше 0, то из S вычитается 1. Если оно меньше или равно 0, то процесс блокируется до тех пор, пока S не станет больше 0, после чего из S вычитается 1.

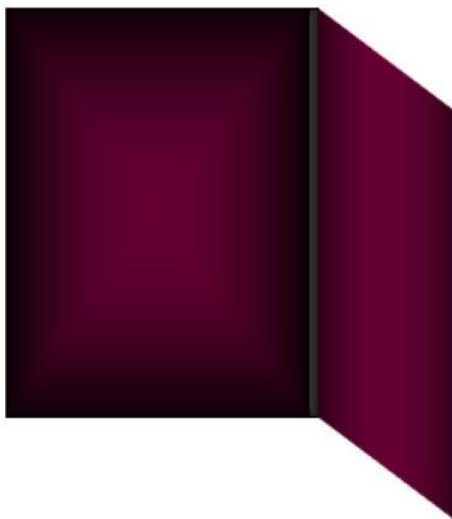
При выполнении операции V над семафором S к его значению просто прибавляется 1. В момент создания семафор может быть инициализирован любым неотрицательным значением.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Функционирование семафора

Семафор $S=0$





ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

С помощью одного экземпляра примитива синхронизации можно обеспечить защиту при обращении к общему ресурсу. Если для защиты одного и того же ресурса используются разные экземпляры примитива синхронизации, то защита, предоставляемая примитивом синхронизации, будет обходиться.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

Несколько примитивов синхронизации .NET являются производными от класса `System.Threading.WaitHandle`, который инкапсулирует собственный дескриптор операционной системы синхронизации и использует механизм сигнализации для взаимодействия потоков. К ним относятся такие классы:

`System.Threading.Mutex`, который предоставляет монопольный доступ к общему ресурсу. Если мьютексом не владеет ни один поток, сообщается состояние мьютекса.

`System.Threading.Semaphore`, ограничивающий число потоков, которые могут одновременно обращаться к ресурсу или пулу ресурсов. Состояние семафора становится сигнальным, когда это число становится больше нуля, и несигнальным, когда равно нулю.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

`System.Threading.EventWaitHandle`, который представляет событие синхронизации потоков и может быть в сигнальном или несигнальном состоянии.

`System.Threading.AutoResetEvent`, который является производным от `EventWaitHandle` и при получении сигнала автоматически сбрасывается в сигнальное состояние после освобождения одиночного потока в состоянии ожидания.

`System.Threading.ManualResetEvent`, который является производным от `EventWaitHandle` и при получении сигнала остается в сигнальном состоянии до вызова метода `Reset`.

Так как `WaitHandle` наследует `System.MarshalByRefObject`, в .NET Framework эти типы можно использовать для синхронизации действий потоков за пределами доменов приложений.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

Семафор позволяет запускать в работу определенное количество потоков, а остальные — приостанавливать, т.е. действует примерно также, как и обычный светофор на дорогах. Например, на экзамене преподаватель приглашает в аудиторию 5 человек. Когда в аудиторию заходят 5 человек, то остальные студенты образуют очередь и на каждого вышедшего из аудитории студента получаем одного входящего. Таким образом, семафор в данном примере контролирует количество студентов в аудитории.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

```
public class Visitor
```

```
{ static Semaphore _semaphore = new Semaphore(5, 5);
```

```
  Thread myThread;
```

```
  // int _count = 1; //счётчик посещений. Один раз - достаточно
```

```
  public Visitor(int i)
```

```
  { myThread = new Thread(Fun);
```

```
    myThread.Name = $"Студент #{i}";
```

```
    myThread.Start();    }
```

Здесь сосредоточена вся функциональность. Так как семафор должны видеть все студенты, то поле `_semaphore` объявлено как `static`.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

```
public class Visitor
```

```
{ static Semaphore _semaphore = new Semaphore(5, 5);  
  Thread myThread;  
  // int _count = 1; //счётчик посещений. Один зайти в аудиторию - достаточно  
  public Visitor(int i)  
  { myThread = new Thread(Fun);  
    myThread.Name = $"«Студент #{i}»";  
    myThread.Start();    }
```

Можно было бы объявить это поле и в методе Main, но я решил унести его в сам класс для того, чтобы он (класс) был самодостаточным. Семафор создается с двумя параметрами (`new Semaphore(5, 5)`) — первый параметр показывает количество свободных мест, а второй — общую емкость. Таким образом мы изначально считаем, что аудитория полностью свободен и три человека могут в него зайти.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

```
public void Fun()  
{  
    _semaphore.WaitOne();  
    Console.WriteLine($" {Thread.CurrentThread.Name} входит в аудиторию");  
    Console.WriteLine($" {Thread.CurrentThread.Name} готовится и сдает");  
    Thread.Sleep(1000); //безудержное веселье  
    Console.WriteLine($" {Thread.CurrentThread.Name} покидает аудиторию");  
    _semaphore.Release();  
}
```

Далее, в методе Fun студенты захватывают семафор (`_semaphore.WaitOne()`), сдают (`Thread.Sleep(1000)`) и, затем, покидают аудиторию, освобождая, тем самым семафор (`_semaphore.Release()`). После чего в аудиторию заходит новый студент и т.д.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

Program

```
{  
    static void Main(string[] args)  
    {  
        for (int i = 1; i < 7; i++)  
        {  
            Visitor visitor = new Visitor(i);  
        }  
    }  
}
```

В методе Main остается только создать некоторое количество пользователей и наблюдать со стороны за их сдачей экзамена.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Операции $P(S)$ и $V(S)$

Стоит отметить, что не всегда студенты (потoki) будут запускаться в том порядке в котором они созданы. То, в какой последовательности будут стартовать потоки — решает операционная система.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Мьютексы

Мьютекс (англ. mutex, от mutual exclusion — взаимное исключение) — одноместный семафор, служащий для синхронизации одновременно выполняющихся потоков.

Мьютексы также представляют собой объекты ядра, используемые для синхронизации, но они проще семафоров, так как регулируют доступ к единственному ресурсу и, следовательно, не содержат счетчиков. По существу они ведут себя как критические секции, но могут синхронизировать доступ потоков разных процессов. Инициализация мьютекса осуществляется функцией `CreateMutex`, для входа в критическую секцию используется функция `WaitForSingleObject`, а для выхода - `ReleaseMutex`.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Мьютексы

Если поток завершается, не освободив мьютекс, последний переходит в свободное состояние. Отличие от семафоров в том, что поток, занявший мьютекс, получает права на владение им. Только этот поток может освободить мьютекс. Поэтому мнение о мьютексе как о семафоре с максимальным значением 1 не вполне соответствует действительности.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Мьютексы

Например, чтобы исключить запись двух потоков в общий участок памяти в одно и то же время, каждый поток ожидает, когда освободится мьютекс, затем при освобождении последнего становится его владельцем и только потом пишет что-либо в этот участок памяти. После всех необходимых действий мьютекс освобождается, предоставляя другим потокам доступ к общему ресурсу.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Мьютексы

В отличие от семафора, который работает в режиме пользователя и блокирует задачи, мьютекс работает в режиме ядра и блокирует сам ресурс. На время выполнения мьютекса, задача, которая захватила мьютекс получает максимальный приоритет, соответственно, если задача выполняет какие-либо действия внутри семафора, то более высокоприоритетная задача может прервать его выполнение, если задача захватила мьютекс, то прекратить выполнение пока она не освободит мьютекс, не может никакая другая задача.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Мьютексы

Класс [System.Threading.Mutex](#) предоставляет монопольный доступ к общему ресурсу. С помощью вызова одной из перегрузок метода [Mutex.WaitOne](#) можно запросить владение мьютексом. [Mutex](#) реализует привязку потока, и поток, который получил мьютекс, должен освободить его, вызвав метод [Mutex.ReleaseMutex](#).

Класс [Mutex](#) может использоваться для межпроцессной синхронизации. Для этого нужно использовать именованный мьютекс, который виден в операционной системе. Чтобы создать экземпляр именованного мьютекса, используйте [конструктор](#) [Mutex](#), который задает имя. Также можно вызвать метод [Mutex.OpenExisting](#), чтобы открыть существующий именованный системный мьютекс.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Мьютексы

Поток вызывает метод [WaitOne](#) из мьютекса, чтобы затребовать владение. Вызов блокируется до тех пор, пока мьютекс не будет доступен или пока не истечет дополнительное время ожидания. Если мьютексом не владеет ни один поток, сообщается состояние мьютекса.

Поток освобождает мьютекс, вызывая его метод [ReleaseMutex](#). Мьютексы поддерживают сходство потоков, т. е. мьютекс может быть освобожден только тем потоком, который им владеет. Если поток освобождает мьютекс, которым он не владеет, в потоке создается исключение [ApplicationException](#).



Мьютексы

Так как класс [Mutex](#) является производным от [WaitHandle](#), вы также можете вызвать статические методы [WaitAll](#) или [WaitAny](#) объекта [WaitHandle](#), чтобы запросить владение [Mutex](#) в сочетании с другими дескрипторами ожидания.

Если поток владеет [Mutex](#), он может указывать этот же [Mutex](#) в повторных запросах ожидания, не прерывая выполнение. Но в этом случае, чтобы отказаться от владения, потребуется освободить [Mutex](#) столько же раз, сколько он был вызван.



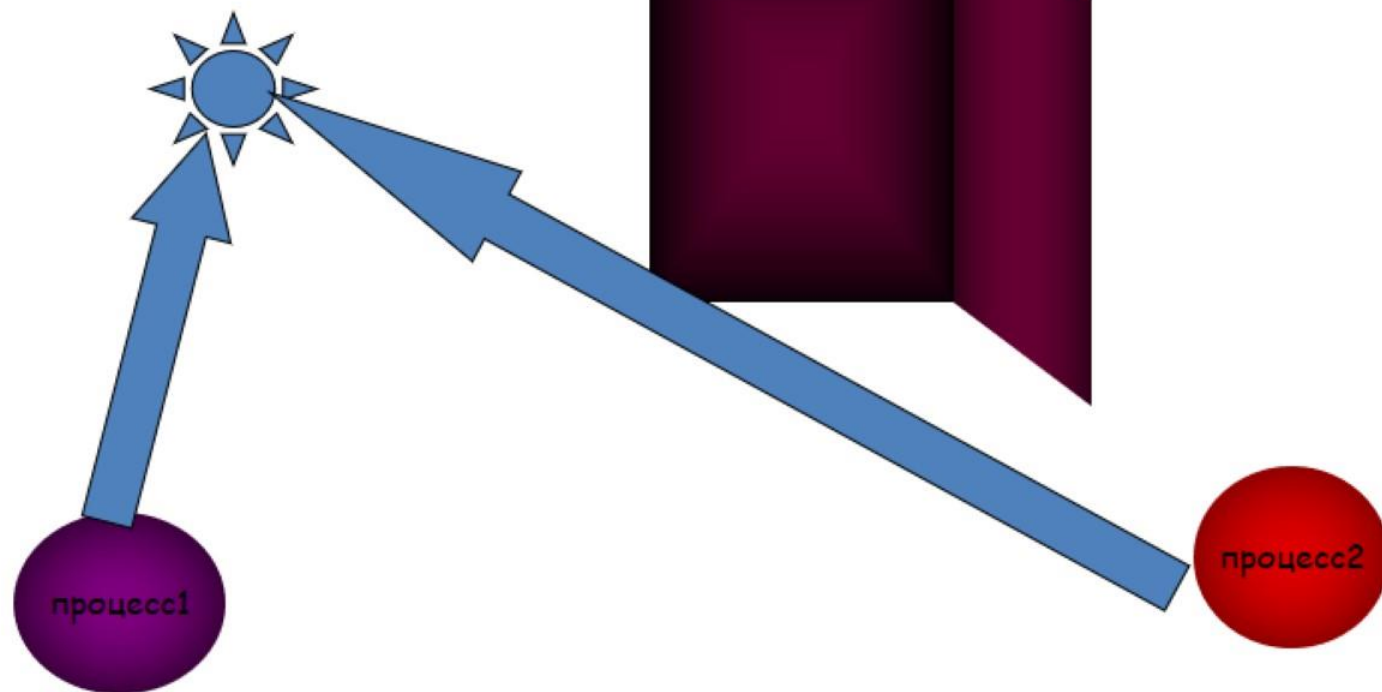
ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Функционирование мьютекса

Ресурс

Свободно

Мьютекс





ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Барьеры

Некоторые приложения делятся на фазы и существует правило, что процесс не может перейти в следующую фазу, пока к этому не готовы все остальные процессы. Этого можно добиться, разместив в конце каждой фазы барьер.

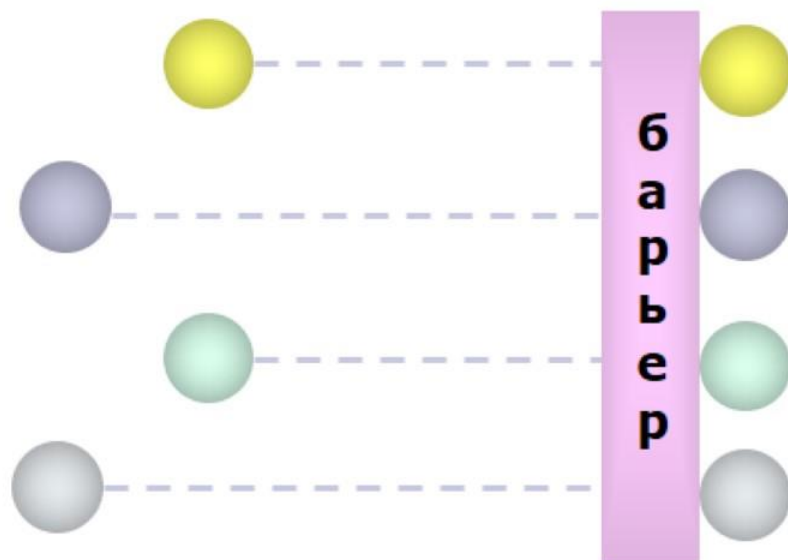
- Когда процесс доходит до барьера, он блокируется до тех пор, пока все процессы не дойдут до барьера.
- Когда процесс завершает вычисления, предусмотренные в одной из фаз, он выполняет примитив Барьер - `barrier()`.

Чаще всего барьер представляет собой библиотечную процедуру и используется в многопроцессорных системах.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Функционирование барьера



Переход через барьер



Мониторы

В 1974 году Хоар и Бринч Хансен предложили примитив синхронизации более высокого уровня, называемый монитором.

Монитор – это набор процедур, переменных и других структур данных, объединенных в особый модуль или пакет.

- Процессы могут вызывать процедуру монитора, но у процедур, объявленных вне монитора, нет прямого доступа к внутренним структурам данных монитора.
- При обращении к монитору в любой момент времени активным может быть только один процесс.
- Монитор не является процессом, это пассивный объект, который приходит в активное состояние только тогда, когда какой-то объект обращается к нему за услугами.

Особенности работы монитора



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Чтобы обеспечить выделение нужного ресурса, процесс должен обратиться к процедуре монитора. Причем в фиксированный момент времени только один процесс может войти в монитор.

- Процессам, которые хотят войти в монитор, когда он уже занят, приходится ждать, причем режимом ожидания управляет сам монитор.
- Внутренние данные монитора могут быть либо глобальными, относящиеся ко всем процедурам монитора, либо локальными (относящиеся только к одной конкретной процедуре монитора).



Особенности работы монитора



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Иногда монитор блокирует обратившийся к нему процесс. Это происходит в случае обращения процесса к занятым ресурсам.

Монитор блокирует процесс с помощью команд `wait()`, а в случае освобождения ресурса выдает команду `signal()`. При этом освободившийся ресурс предоставляется одному из ожидавших его процесса вместе с разрешением на продолжение работы.

Управление передается команде монитора, непосредственно следующей за операцией `wait()`.

Особенности работы монитора



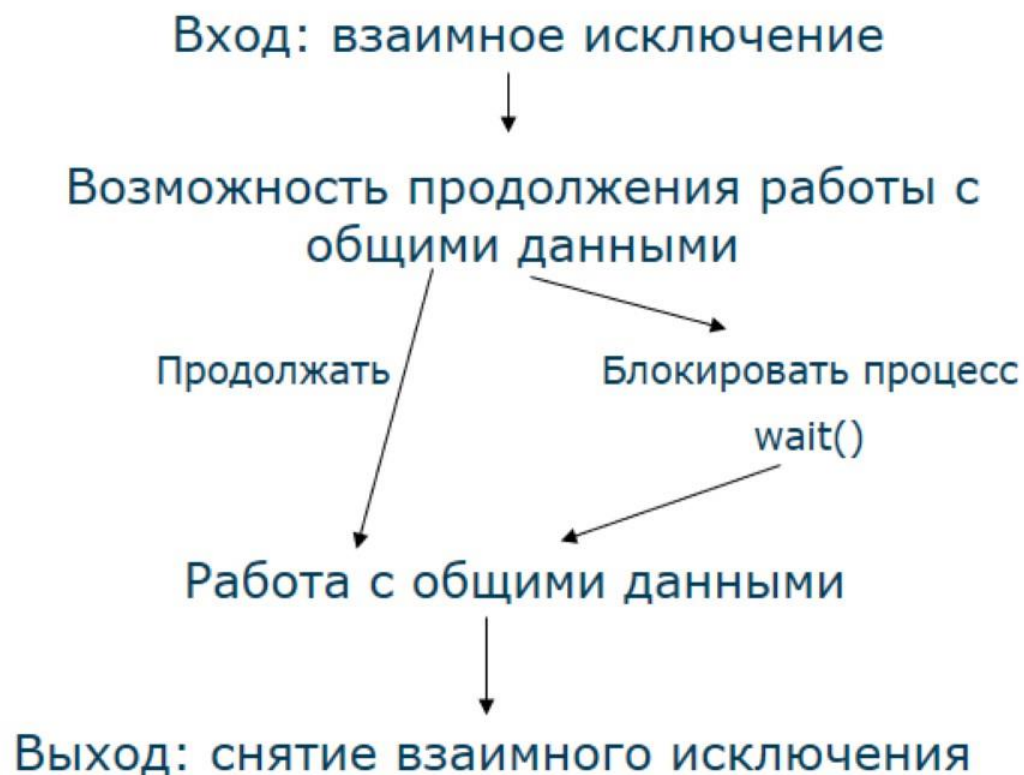
ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Для гарантии того, что процесс, ожидающий некоторый ресурс со временем обязательно его получит, необходимо, чтобы ожидающий процесс имел более высокий приоритет, чем новый процесс, пытающийся войти в монитор.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Процедура монитора





Почтовые ящики

Передача сообщений — это метод межпроцессорного взаимодействия, который позволяет нескольким процессам, выполняющимся как на одном уровне, так и на разных взаимодействующих между собой, путем передачи/приема сообщений.

Для хранения посланного, но еще не полученного сообщения необходимо место. Оно называется буфером сообщений, или почтовым ящиком.



ФГБН
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Почтовые ящики

Почтовый ящик – это информационная структура, поддерживаемая операционной системой. Она состоит из головного элемента, в котором находится информация о данном почтовом ящике, и несколько буферов (гнезд), в которые помещают сообщения. Размер каждого буфера и их количество задаются при образовании почтового ящика.

Реализация почтового ящика требует использования синхронизирующих примитивов, таких как `wait` и `signal`, `send` и `receive`, или каких-либо других средств.

Синхронные и асинхронные примитивы *send и receive*



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Если при взаимодействии двух процессов оба примитива `send` и `receive` являются *блокирующими* (синхронными) говорят, что процессы взаимодействуют синхронно, иначе, если оба примитива *неблокирующие* (асинхронные) - взаимодействие считается асинхронным.

Синхронные и асинхронные примитивы *send и receive*



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

При использовании блокирующего примитива `send` процесс `P1`, послав сообщение процессу `P2`, приостанавливается до момента получения подтверждения о том, что приемник `P2` получил отправленное сообщение. Вызов блокирующего примитива `receive` приостанавливает вызывающий процесс `P2` до того момента, когда он получит сообщение. Синхронное взаимодействие является более простым и надежным.

Синхронные и асинхронные примитивы *send и receive*



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

При использовании неблокирующих примитивов `send` и `receive` управление возвращается процессу `P1`, пославшему сообщение, немедленно, сразу после того, как операционной системе передастся информация о том, где в памяти находится буфер (гнездо), в который нужно поместить сообщение для процесса `P2`. Преимуществом этой схемы является параллельное выполнение вызывающего процесса и процедур передачи сообщения.

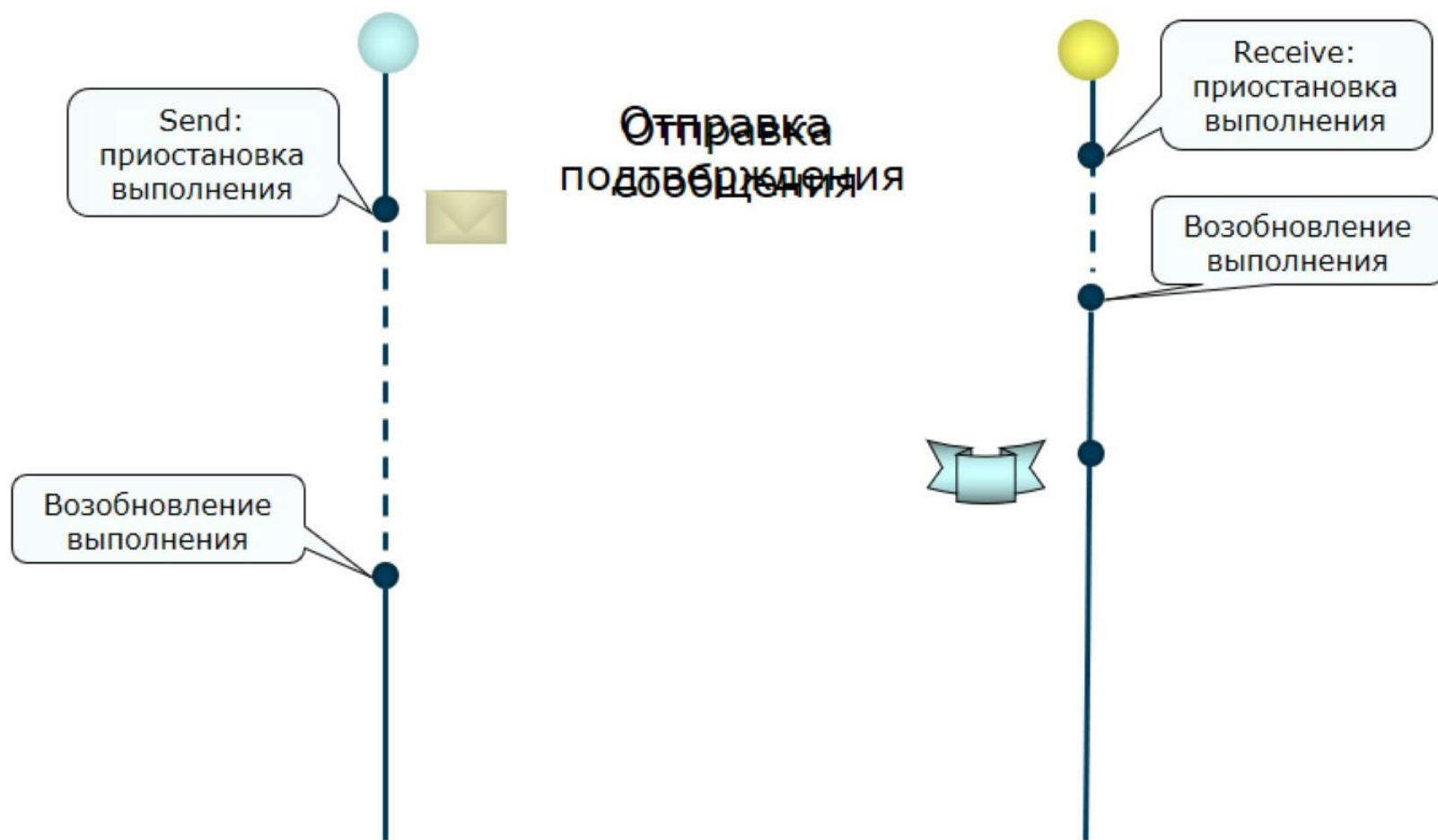
Синхронное взаимодействие



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Выполнение
процесса-отправителя

Выполнение
процесса-получателя



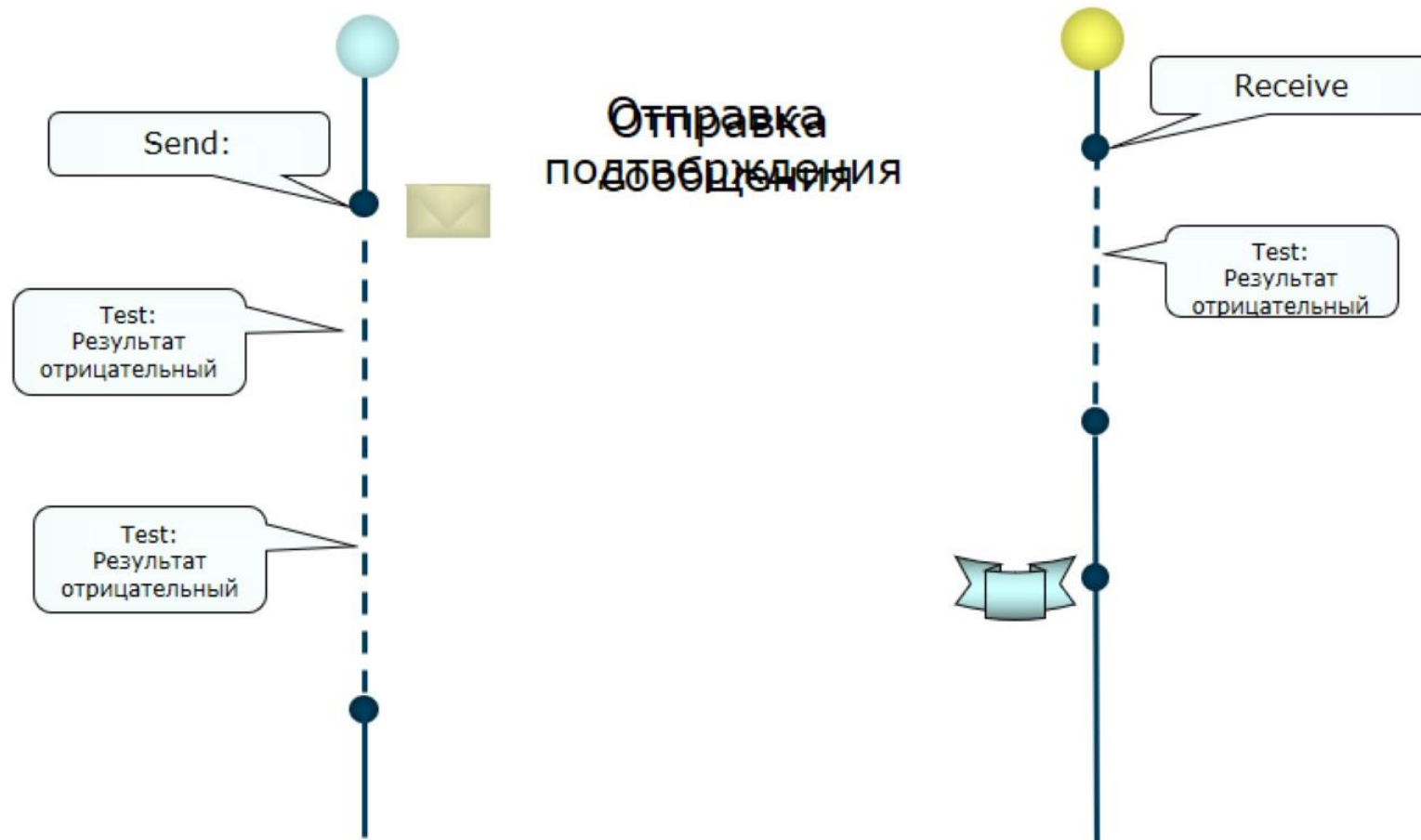
Асинхронное взаимодействие



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Выполнение
процесса-отправителя

Выполнение
процесса-получателя



Работа с почтовыми ящиками



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

• Если процесс $P1$ хочет общаться с процессом $P2$, то $P1$ просит с помощью специальных запросов систему предоставить или образовать почтовый ящик, который свяжет эти два процесса так, чтобы они могли передавать друг другу сообщения.

• Для того, чтобы послать процессу $P2$ какое-то сообщение, процесс $P1$ просто помещает это сообщение в почтовый ящик, откуда процесс $P2$ может в любое время его получить.



Процесс R2 в конце концов обязательно получит сообщение, когда обратиться за ним (если вообще обратиться). Естественно, что процесс R2 должен знать о существовании почтового ящика.

- Так как в системе может быть несколько почтовых ящиков, необходимо обеспечить доступ процессу к конкретному почтовому ящику.

- Если объем передаваемых данных велик, то эффективнее не передавать их непосредственно, а отправлять в почтовый ящик сообщение с информацией о том, где можно найти эти данные.



Правила работы почтовых ящиков

ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Однонаправленные почтовые ящики.

Процесс Р1 может слать сообщения до тех пор, пока имеются свободные гнезда. Если все гнезда заполнены, то Р1 может либо ждать, либо заняться другими делами и попытаться послать сообщение позже. Аналогично процесс Р2 может получать сообщения до тех пор, пока гнезда заполнены, иначе он может либо ждать сообщений, либо продолжать свою работу.



Правила работы почтовых ящиков

ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Двунаправленный почтовый ящик,

связанный с парой процессов, позволяет подтверждать прием сообщений. При наличии множества гнезд каждое из них хранит либо сообщение, либо подтверждение. Чтобы гарантировать передачу подтверждений, когда все гнезда заняты, подтверждение на сообщение помещается в то же гнездо, в котором находится сообщение, и это гнездо уже не используется для другого сообщения до тех пор, пока подтверждение не будет получено.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Функционирование однонаправленного почтового ящика



Почтовый ящик



Запрос на
создание
ПЯ

Процесс Р1

сообщение2

Процесс Р2

Функционирование двунаправленного почтового ящика



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ



Почтовый ящик



Запрос на
создание
пя

Процесс Р1

сообщение1

Подтверждение

Процесс Р2



Достоинства и недостатки почтовых ящиков

Достоинства:
ВЛАДИМИРСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

- Процессу не нужно знать о существовании других процессов до тех пор, пока он не получит сообщения от них;
- Два процесса могут обмениваться больше чем одним сообщением за раз;
- Операционная система может гарантировать, что никакой процесс не вмешается во взаимодействие других процессов;
- Процесс отправитель может продолжать работу не обращая внимания на получателя.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Достоинства и недостатки почтовых ящиков

Недостатки:

- Появление еще одного ресурса, которым надо управлять. Этим ресурсом являются сами почтовые ящики.
- Статический характер этого ресурса: количество буферов для передачи сообщений через почтовый ящик зафиксировано.